Introduction to database system

Dr Mosa Elkhedr

Chapter 1 Outline

- Introduction
- An Example
- Characteristics of the Database Approach
- Actors on the Scene
- Workers behind the Scene
- Advantages of Using the DBMS Approach
- A Brief History of Database Applications
- When Not to Use a DBMS

Overview

- **Traditional database applications**
  - Store textual or numeric information
- **Multimedia databases**
  - Store images, audio clips, and video streams digitally
- **Geographic information systems (GIS)**
  - Store and analyze maps, weather data, and satellite images

Overview (cont'd.)

- **Data warehouses and online analytical processing (OLAP) systems**
  - Extract and analyze useful business information from very large databases
  - Support decision making
- **Real-time and active database technology**
  - Control industrial and manufacturing processes

Introduction

- **Database**
  - Collection of related data
  - Known facts that can be recorded and that have implicit meaning
  - **Miniworld** or **universe of discourse (UoD)**
  - Represents some aspect of the real world
  - Logically coherent collection of data with inherent meaning
  - Built for a specific purpose

Introduction (cont'd.)

- Example of a large commercial database
  - Amazon.com
- **Database management system (DBMS)**
  - Collection of programs
  - Enables users to create and maintain a database
- **Defining** a database
  - Specify the data types, structures, and constraints of the data to be stored

Introduction (cont'd.)

- **Meta-data**
  - Database definition or descriptive information
  - Stored by the DBMS in the form of a database catalog or dictionary
- **Manipulating** a database
  - Query and update the database miniworld
  - Generate reports

Introduction (cont'd.)

- **Sharing** a database
  - Allow multiple users and programs to access the database simultaneously
- **Application program**
  - Accesses database by sending queries to DBMS
- **Query**
  - Causes some data to be retrieved

Introduction (cont'd.)

- **Transaction**
  - May cause some data to be read and some data to be written into the database
- **Protection** includes:
  - System protection
  - Security protection
- **Maintain** the database system
  - Allow the system to evolve as requirements change over time

An Example

- UNIVERSITY database
  - Information concerning students, courses, and grades in a university environment
- **Data records**
  - STUDENT
  - COURSE
  - SECTION
  - GRADE_REPORT
  - PREREQUISITE

An Example (cont'd.)

- Specify structure of records of each file by specifying **data type** for each **data element**
  - String of alphabetic characters
  - Integer
  - Etc.

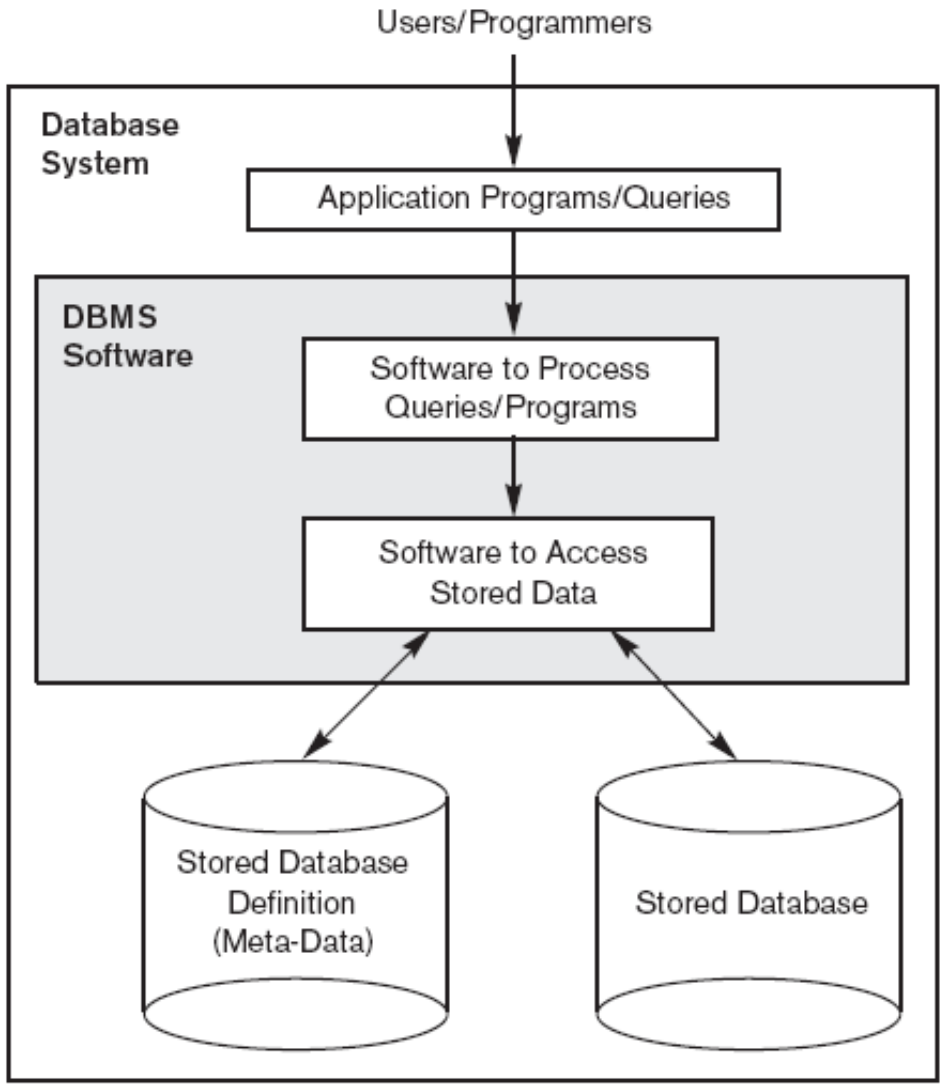


Figure 1.1
A simplified database system environment.

An Example (cont'd.)

- Construct UNIVERSITY database
  - Store data to represent each student, course, section, grade report, and prerequisite as a record in appropriate file
- Relationships among the records
- Manipulation involves querying and updating

An Example (cont'd.)

- Examples of queries:
  - Retrieve the transcript
  - List the names of students who took the section of the ‘Database’ course offered in fall 2008 and their grades in that section
  - List the prerequisites of the ‘Database’ course

An Example (cont'd.)

- Examples of updates:
  - Change the class of ‘Smith’ to sophomore
  - Create a new section for the ‘Database’ course for this semester
  - Enter a grade of ‘A’ for ‘Smith’ in the ‘Database’ section of last semester

An Example (cont'd.)

- Phases for designing a database:
  - **Requirements specification and analysis**
  - **Conceptual design: What data do we need, and how are the data related? It is usually represented using an Entity-Relationship (ER) model/ER diagram (ERD).**
  - **Logical design: is the process of converting the conceptual database design (ER model) into a structure that can be implemented in a database system, usually as relational tables.**
  - **Physical design: is the physical storage details of a particular DBMS.**

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2
A database that stores student and course information.

Characteristics of the Database Approach

- Traditional **file processing**
  - Each user defines and implements the files needed for a specific software application
- Database approach
  - Single repository maintains data that is defined once and then accessed by various users

Traditional File Processing vs. Database Approach

1. Traditional File Processing

Each software application defines and manages its own files.

Files are designed specifically for the requirements of that application.

Different applications may store the same information in separate files.

This approach can lead to data duplication and inconsistency.

Traditional File Processing: University Example

Registration System
Own Student File

→

Library System
Own Student File

→

Accounting System
Own Student File

Problems with Traditional File Processing

Data duplication: the same data may be stored in several files.

Data inconsistency: copies of the same data may become different.

Difficult data sharing: applications may not easily share information.

Update problems: the same change may need to be made in several files.

Program-data dependence: programs can be closely tied to file structures.

2. Database Approach

Data is maintained in a shared repository called a database.

Data is defined and stored centrally rather than separately for every application.

Different users and applications can access the same data.

A Database Management System (DBMS) manages and controls access to the data.

Database Approach: University Example

Registration
Accesses shared data

→

University Database
Student information stored centrally

→

Library / Accounting
Accesses shared data

Example: Updating Student Information

Suppose Ahmed changes his department from Information Systems to Computer Science.

In traditional file processing, the change may need to be made in several application files.

In the database approach, the department is updated in the shared database.

Applications accessing the database can then use the updated information.

Key Concepts to Remember

Traditional file processing: One application → its own files.

Database approach: One shared database → many users/applications.

A DBMS provides centralized management of shared data.

The database approach helps reduce redundancy and improve data consistency and sharing.

Traditional File Processing vs. Database Approach

Aspect	Traditional File Processing	Database Approach
Data storage	Each application has its own files	Shared database
Data duplication	Often high	Reduced
Data sharing	Difficult	Easy among users/applications
Updating	May require multiple files to be updated	Centralized data is easier to update
Program dependence	Programs often depend on file structures	DBMS provides data management
Management	Each application manages its files	DBMS manages the database
Typical use	Simple, isolated applications	Shared, large-scale systems